Univerzita Pardubice. Fakulta Elektrotechniky a Informatiky

Semestrální práce

CourtSy – webová aplikace sportovního klubu

Frontendová SPA aplikace s využitím technologie React

Vypracoval: Daniel Svoboda

Předmět: KTWA2

Vyučující: Ing. Dominik Štursa, Ph.D.

Studijní obor: Webové technologie

Akademický rok: 2026/27

V Třešti dne 23.10.2026

1 Framework a struktura projektu

Aplikace CourtSy je jednostránková aplikace (SPA) fiktivního sportovního klubu postavená na knihovně **React 18** a nástroji **Vite 5**. React jsem zvolil kvůli komponentovému modelu a hooku useSyncExternalStore pro napojení na stav uložený mimo React, Vite kvůli rychlému vývoji a automatickému dělení kódu podle stránek. Směrování zajišťuje **React Router 6**, animace **Framer Motion**. Backend neexistuje, obsah je uložený jako statická data a uživatelská data v localStorage. TypeScript není použitý, datové struktury popisují komentáře JSDoc.

Složka	Obsah
src/views/	obrazovky aplikace (19 komponent pro 21 adres)
src/layouts/	společný rám aplikace (navigace, košík, patička)
src/components/	sdílené komponenty (formulářové pole, odkaz, obrázek, dialog, sekce)
src/lib/	logika bez Reactu (stav, validace, SEO, formátování)
src/data/	obsah webu (členství, areály, programy, akce, zboží)
tests/	jednotkové, komponentové a end-to-end testy

Tabulka 1 Struktura projektu

2 Architektura komponent

Aplikace je poskládaná ve čtyřech vrstvách, kde každá vykresluje tu následující:

1. **Vstup a routování** (main.jsx spustí aplikaci a obalí ji poskytovatelem stavu a routerem, App.jsx obsahuje tabulku všech adres).
2. **Rám aplikace** (RootLayout drží navigaci, košík a patičku a při navigaci se nemění, proto košík přežije přechod mezi stránkami).
3. **Obrazovky** (vždy je zobrazená jedna podle adresy, každá si bere svá data a drží vlastní lokální stav, např. filtr).
4. **Sdílené komponenty** (opakující se stavební prvky jako Field, SmartLink, Img, SelectMenu nebo ConfirmDialog).

Obrazovky se načítají až při otevření (lazy), takže návštěvník úvodní stránky nestahuje kód pokladny. Vzor přehled a detail se opakuje u členství, areálů, programů a akcí. Neznámá adresa zobrazí stránku 404.

3 Správa stavu a dat

Stav je rozdělený podle toho, zda má přežít obnovení stránky:

Stav	Kde je	Úložiště	Po obnovení
košík, profil, objednávky, nastavení	lib/store.js	localStorage	zůstane
otevřený košík	StoreContext	paměť (useState)	zmizí
krok a hodnoty pokladny	Checkout	paměť (useState)	zmizí
filtry, řazení, hledání	jednotlivé obrazovky	paměť (useState)	zmizí
zápisy na prohlídku	PrivateTour	localStorage	zůstane
obsah webu	src/data/	soubory projektu	neměnný

Tabulka 2 Umístění stavu v aplikaci

Komponenty čtou trvalý stav přes jediný hook useStore(), který je na store napojený pomocí useSyncExternalStore. Stav je tak dostupný už při prvním vykreslení a všechny komponenty vidí stejná data. Redux jsem nepoužil, protože stav tvoří jen košík, profil a tři přepínače bez asynchronních operací.

Práce s úložištěm je ošetřená (chyby zachytí try/catch, načtená data se znovu validují a klíče jsou verzované). Vstupy z formulářů ověřují čisté funkce v lib/validation.js (e-mail, telefon, PSČ, platební karta pomocí Luhnova algoritmu) a před uložením se sanitizují. Heslo ani celé číslo karty se neukládá.

4 Nástroje a metody testování

Vrstva	Nástroj	Počet	Co ověřuje
Jednotkové	Vitest	87	validace, store a výpočet součtů, formátování dat
Komponentové	Vitest + React Testing Library	15	formulářové pole, rozbalovací nabídka, pokladna
End-to-end	Playwright (Chromium)	43	navigace, celý nákup, mazání účtu, SEO, stránka 404

Tabulka 3 Přehled automatických testů

Jednotkové testy ověřují logiku ve složce lib/ bez vykreslování komponent. Komponentové testy ovládají formuláře jako uživatel. End-to-end testy běží proti produkčnímu buildu ve skutečném prohlížeči a procházejí celou nákupní cestu až po nalezení objednávky v účtu. Všech 145 testů spouští GitHub Actions před každým nasazením.

Výkon jsem profiloval nástrojem **Lighthouse** nad produkčním buildem v emulaci mobilu. Po optimalizaci (dělení kódu, obrázky ve formátu WebP, písmo v buildu) má výkon měřených stránek 88–92 bodů a přístupnost, Best Practices i SEO 100 bodů.